Drift-Sense: Predicting IMU Dead-Reckoning Drift from Window-Level Motion Features

Yatin Harsith Priyan Sivanandham Pranavi Mathur
GuruManoharGuptaBaratam

Mid-semester report

1 Dataset and Exploratory Analysis

1.1 EuRoC MAV

We use three Machine Hall sequences from the EuRoC MAV dataset [2]: MH_01_easy, MH_02_easy, and MH_03_medium. Each flight is a few minutes of a real hexacopter with a stereo camera and an ADIS16448 IMU. Phase 1 uses only `imu0/data.csv` (gyroscope in rad/s, accelerometer specific force in m/s^2, nominally 200 Hz) and `state_groundtruth_estimate0/data.csv` (position, Hamilton quaternion, velocity, and IMU biases). Camera images are reserved for Phase 2.

1.2 Data quality

Table 1 summarises the three streams after loading. There are no missing values. Both IMU and the provided ground-truth estimate are sampled at 200 Hz with a 5 ms median step and negligible jitter. Nearest-timestamp alignment drops 1.17%, 1.32%, and 2.58% of IMU rows (MH_01–MH_03), almost entirely at sequence edges where ground-truth coverage starts later than the IMU. We treat those drops as coverage gaps, not sensor faults.

Table 1: Sequence-level quality checks.

Sequence	IMU rows	Duration (s)	Rate (Hz)	Missing	Align drop
MH_01_easy	36820	184.1	200	0	1.17%
MH_02_easy	30400	152.0	200	0	1.32%
MH_03_medium	27008	135.0	200	0	2.58%

Raw gyro over the first 10 s of MH_01 is small-amplitude chatter ($\sim$0.2 rad/s), i.e. noise plus slow attitude change rather than aggressive spins. Accelerometer magnitude stays near g; the vehicle is pitched, so gravity appears on a_x and a_z together, not on a_z alone.

1.3 Easy versus medium

Ground-truth speed distinguishes the labels: mean 0.44 m/s and 0.49 m/s on the two easy flights versus 0.99 m/s on MH_03 (peaks 2.8 m/s). Path length is $\sim$81 m and $\sim$74 m versus $\sim$131 m. Mean gyro magnitude is also higher on MH_03 (0.29 vs 0.21 rad/s). The 3-D trajectories occupy a similar machine-hall volume; MH_03 is "medium" because of speed and turning, not a larger room.

1.4 Drift label

The headline target is the 1 s raw-IMU position error y = `err_mag_m`. Across 724 / 596 / 523 overlapping windows the means are 0.186, 0.186, and 0.181 m with standard deviations 0.012, 0.013, and 0.021 m. The easy-sequence histograms are tight and slightly left-skewed; MH_03 is a little wider (max 0.30 m) but still not a long tail. We therefore **do not** log-transform the primary target.

The same windows after subtracting the dataset's ground-truth IMU biases (`err_mag_m_bc`) have means 0.021, 0.018, and 0.030 m. MH_03 then shows the most residual structure (max 0.176 m, skew 2.3). This supports the claim that 1 s raw drift is bias-dominated; the bias-corrected series is an oracle comparison, not the leaderboard target. Signed axis errors $(\Delta x, \Delta y, \Delta z)$ vary more than the Euclidean magnitude, so Phase 1 models should also be trained on the 3-axis target.

Mean error versus window length (0.25–2.0 s) grows superlinearly for the raw integrator ($\sim$6 mm at 0.25 s to $\sim$1.2 m at 2 s) and remains much smaller after bias correction. The primary window stays 1.0 s / 0.25 s stride as specified.

1.5 Train/test protocol

Windows overlap by 75%. A random split of rows would place nearly identical IMU slices in both train and test and inflate R^2. We use leave-one-sequence-out, recorded in `data/processed/splits.json`: train on two flights, test on the third, rotated three times. The leakage test `tests/test_no_leakage.py` asserts that no sequence appears on both sides of any fold. The primary target in that file is `err_mag_m`.

2 Method

2.1 Drift Label Construction

The EuRoC dataset does not provide a pre-computed odometry-drift signal; it supplies raw IMU measurements and precise ground-truth pose from a motion-capture system. We construct the regression target — the position error that a Visual-Inertial Odometry system would accumulate over a short time window — directly from these two streams.

2.1.1 Temporal Alignment

IMU measurements (200 Hz) and ground-truth pose estimates are recorded asynchronously. For every IMU sample we attach the temporally nearest ground-truth sample, discarding IMU samples whose nearest match exceeds a 20 ms tolerance. Across the three sequences used in Phase 1 (MH_01_easy, MH_02_easy, MH_03_medium), this discarded between 1.2% and 2.6% of IMU samples, concentrated at sequence boundaries where ground-truth coverage is sparser than IMU coverage.

2.1.2 Windowing

The aligned stream is divided into overlapping windows of $T_w = 1.0$ s (200 IMU samples at 200 Hz), with a stride of $T_s = 0.25$ s between consecutive window starts. This yielded 724, 596, and 523 windows for MH_01_easy, MH_02_easy, and MH_03_medium respectively.

2.1.3 IMU Strapdown Integration

For each window, we compute a model-free position estimate by integrating the raw IMU measurements forward using standard strapdown inertial mechanization. Starting from the true

position p_0, velocity v_0, and orientation quaternion q_0 at the window's first timestamp (taken directly from ground truth), we integrate at each IMU sample i with timestep Δt_i:

$$a_{\text{world},i} = R(q_i)\, a_{\text{body},i} + g \tag{1}$$

$$v_{i+1} = v_i + a_{\text{world},i}\, \Delta t_i \tag{2}$$

$$p_{i+1} = p_i + v_i \Delta t_i + \tfrac{1}{2} a_{\text{world},i} \Delta t_i^2 \tag{3}$$

$$q_{i+1} = q_i \otimes \Delta q(\omega_i, \Delta t_i) \tag{4}$$

where $R(q_i)$ is the rotation matrix corresponding to the current orientation estimate, g is the world-frame gravity vector, and $\Delta q(\omega_i, \Delta t_i)$ is the incremental rotation from the gyroscope reading ω_i.

Two sign/frame conventions — the sign of g and whether $R(q_i)$ maps body-to-world or world-to-body — are not fixed by the dataset documentation alone and were determined empirically: we evaluated all four combinations on a short (0.25 s) probe window against ground truth and selected the combination minimizing position error. This yielded $g = (0, 0, -9.81)\,\text{m/s}^2$ under a body-to-world rotation convention, with a probe-window error of 0.017 m — one to two orders of magnitude lower than the three alternative conventions (0.27–0.63 m), which we take as strong evidence the correct convention was identified rather than an arbitrary best-of-four selection.

2.1.4 Label Definition

The drift label for a window is the Euclidean distance between the integrated position estimate $\hat{p}_{\text{end}}$ at the window's final timestamp and the true position p_{end} from ground truth:

$$y_{\text{err}} = \|\hat{p}_{\text{end}} - p_{\text{end}}\|_2 \tag{5}$$

The signed per-axis error $(\Delta x, \Delta y, \Delta z)$ is retained alongside the scalar magnitude for use in the Phase 2 trajectory correction step and as additional Phase 1 regression targets. Across the three sequences, mean 1-second-window *raw*-IMU drift was consistent, ranging from 0.181 m to 0.186 m, with maximum single-window error of 0.30 m on MH_03_medium. After oracle bias subtraction the same 1 s windows have mean residual 0.018–0.030 m (MH_03 still the largest).

The same windows are also integrated after subtracting the dataset's ground-truth IMU bias estimates. The resulting columns (`err_mag_m_bc`, `err_dx/dy/dz_bc`) are an oracle comparison, not a replacement of the approved raw-IMU target: a deployed estimator is not handed those biases, and all features remain raw IMU. Whether the residual is more motion-dependent is measured in the EDA, not assumed.

2.1.5 Validation

We verified the label construction is physically sound by confirming that mean drift error increases with window length (0.25 s to 2.0 s), consistent with the expected accumulation behavior of open-loop inertial integration. This check, along with the convention-selection probe error, is automated in `tests/test_label_sanity.py` and passed on all sequences prior to any modeling work.

2.2 Feature Engineering

Each window is a 200×6 matrix of raw IMU samples (three gyroscope and three accelerometer axes over one second). Classic regressors cannot consume this raw tensor directly, so we summarise every window as a single fixed-length vector of 104 hand-crafted features. The guiding hypothesis is that open-loop inertial integration accumulates position error fastest in windows that are *energetic*, *erratic*, or *abrupt*; the feature families below are designed to quantify exactly those properties from the signal alone, using only raw IMU (no ground-truth information leaks in).

2.2.1 Time-domain features (60)

For each of the six channels we compute ten statistics: mean, standard deviation, variance, RMS, minimum, maximum, peak-to-peak range, skewness, excess kurtosis, and zero-crossing rate. The first-moment and energy terms (mean, RMS, variance) measure how much the drone moved and how large a bias the integrator must contend with; the shape terms (skewness, kurtosis) flag asymmetric or spike-laden windows; and the zero-crossing rate, computed about each channel's own mean so the accelerometer gravity offset does not dominate, is a cheap proxy for oscillation frequency. This family contains the single strongest linear correlate of the label, the accelerometer x-axis mean and RMS ($|r| \approx 0.47$), and supplies the top-ranked feature under a RandomForest importance analysis (`ax_rms`).

2.2.2 Frequency-domain features (18)

Time-domain statistics describe *how much* a window moves but not *how* it moves. For each channel we remove the per-window mean, take a real FFT, and extract the dominant frequency, the total spectral energy, and the normalised spectral entropy of the power spectrum. Mean removal is essential: without it the accelerometer's large gravity/DC term would be read as a spurious 0 Hz peak. Spectral entropy distinguishes clean, tonal motion (energy in one band) from broadband, noise-like windows; the per-axis spectral-entropy features are among the strongest label correlates ($|r| \approx 0.31$–0.37), consistent with noise-like windows drifting more than smoothly manoeuvring ones.

2.2.3 Cross-axis and magnitude features (26)

A drone is a rigid body, so its axes are coupled; three groups of features capture coupling the per-axis statistics miss. Accelerometer and gyroscope *vector magnitudes* (mean, standard deviation, maximum of $\|a\|$ and $\|\omega\|$) give an orientation-independent measure of motion, invariant to the drone's heading. *Jerk* features (per-axis RMS and the mean and peak magnitude of $\mathrm{d}a/\mathrm{d}t$) target abrupt changes, which are precisely where discrete strapdown integration between samples is least accurate. Finally, the fifteen pairwise Pearson *correlations* between channels describe coordinated motion: strong cross-axis correlation is the signature of a banking turn or climb rather than uncorrelated sensor noise, and several accelerometer–gyroscope cross terms (e.g. `corr_gz_ax`) rank highly under RandomForest importance.

2.2.4 Scaling and the no-leakage constraint

Because the project splits by sequence (leave-one-sequence-out, Section 2.1), feature standardisation must respect fold boundaries: a `StandardScaler` is fit on each fold's training sequences only and then applied to the held-out sequence. Fitting a scaler across all data first would leak test-set statistics into training. The handoff tables `features_<seq>.csv` are therefore stored *unscaled*, and scaling is performed per fold at modelling time.

2.2.5 Redundancy and feature selection

The feature set is deliberately broad and contains known collinearities (variance is the square of standard deviation; RMS approximates the absolute mean when the mean dominates). Pooled across the three sequences, 17 feature pairs exceed a 0.95 absolute correlation. Tree-based models are insensitive to this redundancy, and a feature-selection experiment (notebook `04_feature_engineering`) confirms that a compact subset of the highest-ranked features retains most of the predictive signal; the full set is retained for Phase 1 so that per-family contributions can be compared honestly in the ablation discussion.

3 Models

Every model maps a window's feature vector $\mathbf{x} \in \mathbb{R}^{104}$ (Section 2.2) to a predicted drift $\hat{y}$, the 1 s position error `err_mag_m` in metres. We compare a mean baseline, four linear models (OLS, Ridge, Lasso, and a linear model trained by mini-batch gradient descent) and one non-linear model, a Random Forest. All hyperparameters are chosen by grid search (Section 3.4).

3.1 Baseline and linear models

Mean baseline. Predict the average training label for every window, $\hat{y} = \bar{y}_{\text{train}}$. It ignores the features entirely, so any model that cannot beat it has learned nothing that transfers to a new flight.

Linear regression (OLS). Ordinary least squares: the linear model $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$ that minimises the squared training error $\sum_i (y_i - \mathbf{w}^\top \mathbf{x}_i - b)^2$, with nothing to tune. It is the unregularised reference for Ridge: comparing the two measures what Ridge's penalty buys rather than assuming it (Section 4.2).

Ridge regression. The same linear model, fitted by minimising

$$\sum_i \big(y_i - \mathbf{w}^\top \mathbf{x}_i - b\big)^2 + \alpha \|\mathbf{w}\|^2. \tag{6}$$

The penalty $\alpha\|\mathbf{w}\|^2$ shrinks the weights. It is needed here because many features are near-duplicates (e.g. variance is the square of standard deviation; 17 feature pairs have $|r| > 0.95$): without it, a least-squares fit would give such pairs large weights of opposite sign that cancel on the training data and blow up on new data (as OLS does, Section 4.2). Features are standardised (zero mean, unit variance) first, so the penalty treats every feature equally. We tune $\alpha \in \{10^{-2}, 10^{-1}, \ldots, 10^{5}\}$ (8 values).

Lasso regression. The same model with an L1 penalty instead,

$$\frac{1}{2n} \sum_i \big(y_i - \mathbf{w}^\top \mathbf{x}_i - b\big)^2 + \alpha \|\mathbf{w}\|_1, \qquad \|\mathbf{w}\|_1 = \textstyle\sum_j |w_j|. \tag{7}$$

L2 shrinks every weight a little; L1 pushes weak weights to *exactly* zero, so Lasso also selects features. With redundant features this is a natural fit: from a group of near-duplicates it tends to keep one. The useful range of α is small because the label's spread is only about 0.015 m; we tune $\alpha \in \{10^{-6}, 10^{-5}, \ldots, 10^{-1}\}$ (6 values).

3.2 Mini-batch gradient descent

OLS, Ridge and Lasso are fitted by a solver. As a fourth linear model we fit the same $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$ iteratively, with our own implementation of mini-batch gradient descent. Each epoch shuffles the training windows and splits them into batches B; for each batch the weights take one step against the gradient of the batch loss

$$L_B(\mathbf{w}, b) = \frac{1}{2|B|} \sum_{i \in B} \big(y_i - \mathbf{w}^\top \mathbf{x}_i - b\big)^2 + \alpha R(\mathbf{w}), \qquad \mathbf{w} \leftarrow \mathbf{w} - \eta \, \nabla_{\mathbf{w}} L_B, \tag{8}$$

where $R(\mathbf{w}) = \frac{1}{2}\|\mathbf{w}\|^2$ (L2) or $\|\mathbf{w}\|_1$ (L1, using the subgradient $\operatorname{sign}(\mathbf{w})$), and η is the learning rate. Batch size sits between stochastic gradient descent (one window per step) and full-batch gradient descent (all windows per step). Averaging the gradient over a batch smooths out

single unusual windows while still taking many steps per epoch; notebook 05 shows that at the same learning rate, batch size 1 diverges on our data (some standardised features have extreme outliers) and the full batch barely moves in 50 epochs, while batches of 32 converge. Features are standardised first, and weights start at zero with b at the training mean. We run 50 epochs and tune $\eta \in \{0.001, 0.01\}$, batch size $\in \{32, 128\}$, the penalty (L2 or L1) and $\alpha \in \{10^{-4}, \ldots, 10^{-1}\}$: 32 combinations.

3.3 Random Forest

A *regression tree* recursively splits the data with yes/no questions on one feature at a time ("is `ax_rms` > 9.6?"). At each node it picks the feature and threshold that most reduce the squared error of the label in the two resulting groups, and a leaf predicts the mean label of the training windows that reach it. A single deep tree fits noise. A *Random Forest* [1] averages B trees,

$$\hat{y}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^{B} T_b(\mathbf{x}), \tag{9}$$

where each tree T_b is trained on a bootstrap resample of the windows and considers only a random subset of features at each split. These two sources of randomness make the trees' errors less correlated, so averaging them cuts variance while keeping each tree's low bias.

Trees depend only on the *order* of a feature's values, so no scaling is needed. We use $B = 300$ trees and tune the maximum depth ($\{\text{none}, 4, 8\}$), the minimum windows per leaf ($\{1, 5, 20\}$) and the number of features tried per split (all $p = 104$, $\sqrt{p}$, or $0.3p$): 27 combinations. Shallow trees with large leaves are the forest's main defence against over-fitting on our small data set.

3.4 Evaluation protocol

Windows overlap by 75% (1 s length, 0.25 s stride), so neighbouring windows are nearly identical. A random train/test split would therefore score a model on near-copies of its training windows. We split by *sequence* instead, using *nested* leave-one-sequence-out (LOSO) cross-validation:

- **Outer loop** (`splits.json`): three folds, each training on two sequences and testing on the third. Only these held-out scores are reported.
- **Inner loop** (`tuning.py`): within a fold's two training sequences, each hyperparameter combination is trained on one sequence and scored on the other, both ways. The best combination by mean absolute error is refit on both and then scored once on the outer test sequence.

Every scaler sits inside its model's pipeline, so it is fit on training windows only. Notebook 05 verifies this for each fold: each saved scaler's mean equals the training-sequence mean, not the pooled mean. As a further control, a Random Forest trained on *shuffled* labels does no better than the mean baseline on held-out data.

We report mean absolute error (MAE), root mean squared error (RMSE), and $R^2 = 1 - \sum_i (y_i - \hat{y}_i)^2 / \sum_i (y_i - \bar{y})^2$ computed on the held-out sequence. $R^2 = 0$ means no better than knowing the *test* sequence's mean. The mean baseline uses the *training* mean, so its R^2 is slightly negative.

4 Results

All scores are on the held-out sequence of each leave-one-sequence-out fold (Section 3.4). The primary target is `err_mag_m`.

Table 2: Held-out MAE (m) for `err_mag_m`, by test sequence.

Model	MH_01_easy	MH_02_easy	MH_03_medium	Mean
Mean baseline	0.0094	0.0101	0.0145	0.0113
Linear (OLS)	0.0088	0.0081	0.0175	0.0115
Ridge (L2)	0.0077	0.0074	0.0151	0.0100
Lasso (L1)	0.0076	0.0075	0.0147	0.0099
Mini-batch GD	0.0075	0.0075	0.0151	0.0100
Random Forest	0.0077	0.0078	0.0151	0.0102

Table 3: Held-out R^2 for `err_mag_m`, by test sequence.

Model	MH_01_easy	MH_02_easy	MH_03_medium	Mean
Mean baseline	−0.037	−0.016	−0.047	−0.033
Linear (OLS)	0.103	0.344	−0.701	−0.084
Ridge (L2)	0.312	0.483	−0.057	0.246
Lasso (L1)	0.335	0.463	−0.025	0.258
Mini-batch GD	0.344	0.463	−0.058	0.250
Random Forest	0.312	0.405	−0.049	0.223

4.1 Held-out performance

Tables 2 and 3 give the headline result. On the two *easy* sequences every model beats the mean baseline. For the regularised linear models (Ridge, Lasso, mini-batch GD) and Random Forest, MAE falls by 18–27% (e.g. Ridge on MH_02: 0.0101 → 0.0074 m), and the models explain 31–48% of the test sequence's label variance; unregularised OLS gains less (Section 4.2). On *MH_03_medium* no model beats the baseline: the tuned models reach MAE 0.0147–0.0151 m against the baseline's 0.0145 m, with R^2 slightly negative, and OLS is clearly worse (0.0175 m, $R^2 = -0.70$). Averaged over folds, Lasso (0.0099 m), Ridge and mini-batch GD (0.0100 m) and Random Forest (0.0102 m) are within 0.0003 m of each other. With only three test flights that difference is within noise, so we treat these four models as tied rather than naming a single best one; the inner-CV scores, which never see the test flight, tell the same story. A regularised linear model does as well as the non-linear forest on this data. RMSE tells the same story (fold means: baseline 0.0154, OLS 0.0162, Ridge 0.0134, Lasso 0.0133, mini-batch GD 0.0134, Random Forest 0.0136 m).

Tuning chose strong shrinkage for Ridge ($\alpha = 1000$ in every fold, inside a grid that runs to 10^5) and $\alpha = 10^{-3}$ for Lasso in every fold, also inside its grid. For mini-batch GD it chose the L1 penalty in every fold. For Random Forest it chose shallow or large-leaf trees in two of the three folds. All these choices point to the same thing: the signal is weak and over-fitting is the main risk.

4.2 Over-fitting

A model over-fits when it learns the noise of its training flights instead of a relationship that carries over to a new one. The symptom is a small error on the training windows and a much larger one on the held-out sequence. We therefore also score every model on the windows it was fitted on (`train_mae`) and compare.

Without a penalty, the linear model over-fits. OLS fits its training flights slightly *better* than Ridge (fold-mean training MAE 0.0083 against 0.0086 m), but does worse on every held-out flight (Table 2). Averaged over folds it scores 0.0115 m, worse even than the mean baseline (0.0113 m), while Ridge scores 0.0100 m. The damage is largest on MH_03_medium, where OLS

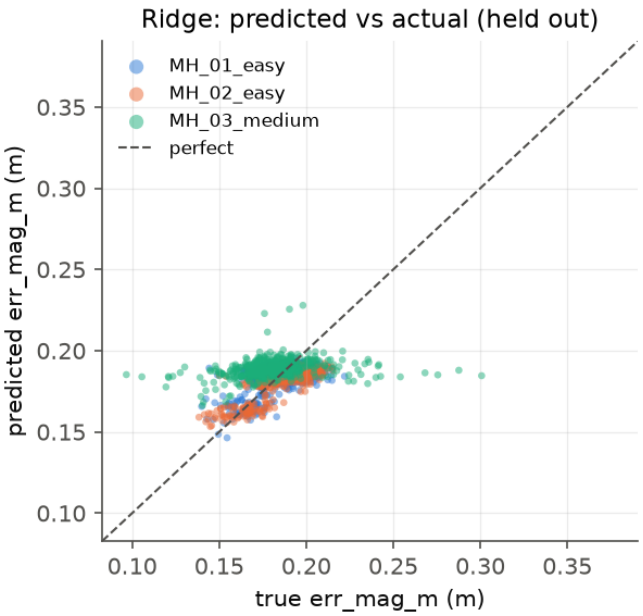


Figure 1: Ridge: predicted vs. true drift on each held-out sequence. Predictions cluster in a narrow band around 0.18–0.19 m: the model captures the level, not the extremes. The other models look the same (one figure per model in `figures/ml/` and notebook 06).

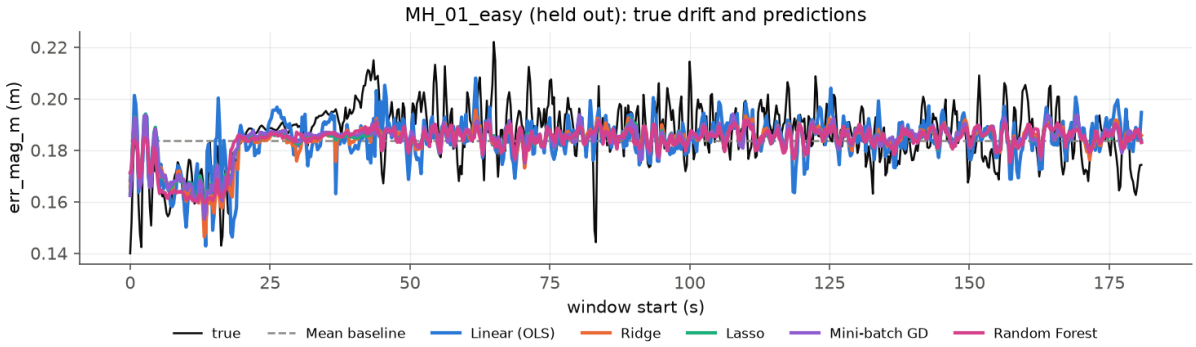


Figure 2: True drift and model predictions over the held-out MH_01_easy flight.

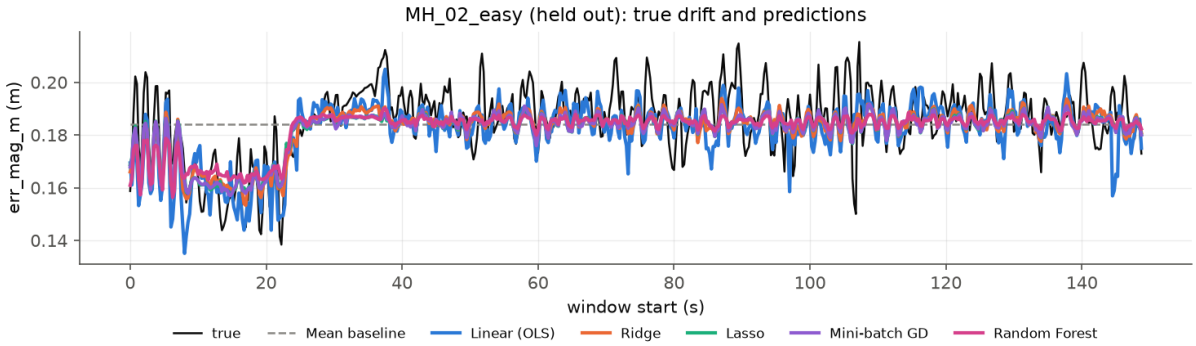


Figure 3: True drift and model predictions over the held-out MH_02_easy flight.

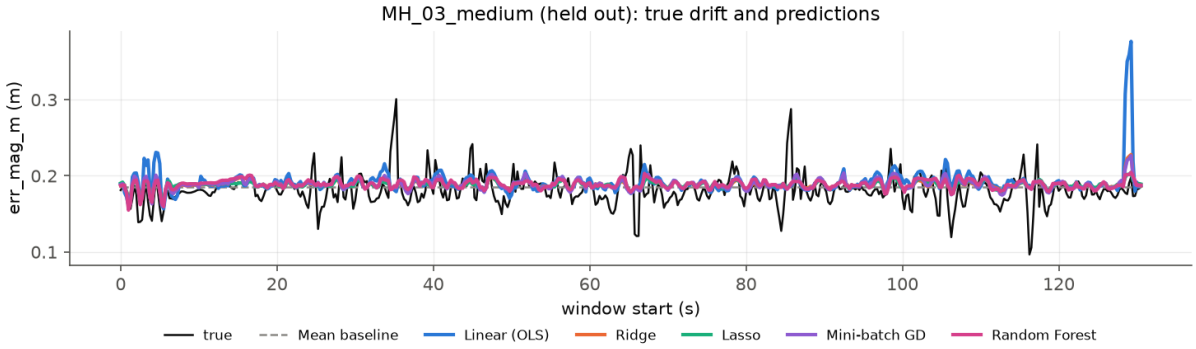


Figure 4: True drift and model predictions over the held-out MH_03_medium flight.

reaches $R^2 = -0.70$. The inner cross-validation saw the same thing before any test data was touched: OLS's inner-CV MAE is 0.045, 0.028 and 0.008 m in the three folds, against 0.012, 0.012 and 0.007 m for Ridge. This is the failure the L2 penalty is there to prevent (Section 3): with many near-duplicate features, least squares assigns large weights of opposite sign that cancel on the training flights and not on a new one. Lasso's L1 penalty goes further and sets all but 8–11 of the 104 weights to exactly zero in each fold (notebook 05); the features it keeps (`ax_mean`/`ax_rms`, `acc_mag_mean`, `az_spec_entropy`, `gy_mean`) overlap with those that permutation importance ranks highest below.

Every flexible model can memorise; tuning stops it. To see what the tuning protects against, we refit each model with the most flexible setting in its own grid, the setting best able to memorise the training windows, and compare it with the tuned model (Table 4). With almost no

Table 4: Training vs. held-out MAE (m) for `err_mag_m`, averaged over folds, for the tuned model and its most flexible grid setting. Ratio is held-out MAE over training MAE (mean over folds). Loose mini-batch GD: L2, $\alpha = 10^{-4}$, $\eta = 0.01$, batch size 32.

Model	Setting	Train MAE	Held-out MAE	Ratio
Ridge (L2)	tuned ($\alpha = 1000$)	0.0086	0.0100	1.3
	loose ($\alpha = 0.01$)	0.0083	0.0114	1.5
Lasso (L1)	tuned ($\alpha = 10^{-3}$)	0.0089	0.0099	1.2
	loose ($\alpha = 10^{-6}$)	0.0083	0.0114	1.5
Mini-batch GD	tuned	0.0087	0.0100	1.3
	loose (see caption)	0.0084	0.0107	1.4
Random Forest	tuned	0.0076	0.0102	1.7
	loose (unlimited depth, leaf 1)	0.0028	0.0102	4.0

penalty, Ridge and Lasso both behave like OLS: training error drops slightly and held-out error rises to 0.0114 m (from 0.0100 and 0.0099 m tuned). Mini-batch GD with its weakest penalty and largest step does the same on a smaller scale (0.0107 against 0.0100 m). (The loosest Lasso, $\alpha = 10^{-6}$, reaches the solver's iteration limit before fully converging; it is only used here, where it behaves like OLS as expected.) The loose Random Forest memorises much harder (training MAE 0.0028 m) but its held-out MAE does not move: averaging 300 decorrelated trees cancels much of what the individual deep trees memorise. In every case inner cross-validation rejected the loose setting, so the reported models are the ones with the smallest gap.

The remaining gap on MH_03 is distribution shift, not over-fitting. On the two easy folds every tuned model's held-out MAE is *below* its training MAE (ratios 0.76–0.87), because

those training sets include the harder MH_03 flight. When MH_03 is held out, the ratio jumps to 2.1–3.3 for Ridge, Lasso, mini-batch GD and Random Forest. But the mean baseline, which has no parameters to over-fit, also jumps to 1.55 (0.0093 → 0.0145 m). A gap that appears even for a constant predictor comes from the test flight being different, not from the model memorising; regularisation cannot close it. Section 4.3 examines this shift.

4.3 Failure analysis

Gains on easy flights come mostly from the opening segment. On MH_01 and MH_02 the first 20 s sit at a lower drift level (mean true label 0.1669 and 0.1686 m) than the rest of the flight (0.1883 and 0.1891 m; Figures 2 and 3). The features pick this up. In that segment Random Forest cuts the baseline's MAE from 0.0173 to 0.0080 m (MH_01) and from 0.0193 to 0.0104 m (MH_02). After 25 s the gain is much smaller: 0.0086 → 0.0078 and 0.0083 → 0.0073 m. The models track the general drift level but not its second-to-second fluctuations.

MH_03_medium is out of distribution. When MH_03 is held out, the models train only on the two easy flights. MH_03 rotates faster: its mean `gyro_mag_mean` is 0.295 rad/s against 0.213 in training, and 4.8% of its windows fall outside the training range. Its label is also much more spread out (std 0.0205 m against 0.0124 m in training) and spikes to 0.301 m, with 5.2% of its windows outside the training label range. The models have never seen this regime. Error also broadly grows with rotation rate: in the fastest-rotating quarter of windows, Random Forest's MAE is 1.41×, 1.25× and 1.63× its value in the slowest quarter (MH_01, MH_02, MH_03).

Most of the remaining error sits in drift spikes. The worst 10% of windows carry 28%, 27% and 32% of Random Forest's total absolute error (MH_01, MH_02, MH_03). The correlation between prediction and truth falls from 0.60 and 0.70 on the easy flights to 0.31 on MH_03. A constant offset explains only 6%, 10% and 14% of the squared error. The failure is therefore mostly not a calibration shift that adding a constant would fix. The models simply cannot anticipate sudden jumps in drift from window-level summary statistics.

Which features matter. Shuffling one feature on held-out data hurts most for the accelerometer level features. For Random Forest these are `ax_mean` (+0.244 mm MAE) and `ax_rms` (+0.159 mm); for Ridge they are `ax_mean` (+0.321 mm), `acc_mag_mean` (+0.319 mm) and `ax_rms` (+0.293 mm) (Figure 5). This matches the feature analysis of Section 2.2. It also fits the label being driven mainly by how gravity and accelerometer bias project into the integration, not by vigorous motion as such.

4.4 Bias-corrected target (oracle comparison)

Running the same pipeline on `err_mag_m_bc`, the drift left after subtracting the dataset's own bias estimates, the best fold-mean MAE is 0.0109 m (Random Forest) against a baseline of 0.0122 m. R^2 never exceeds 0.069 (Lasso on MH_01) on any fold and is negative when averaged over folds for every model. Once bias is removed, the remaining drift is barely predictable from these motion features.

4.5 Summary

The classic models learn a real but modest relationship between IMU window statistics and drift. That relationship transfers between the two easy flights, but not to the faster MH_03_medium flight. The obstacles are distribution shift and unpredictable drift spikes, not the choice of model. Over-fitting is real but under control: unregularised OLS and the loosest grid settings memorise the training flights and lose on new ones, and the sequence-grouped tuning picks settings whose

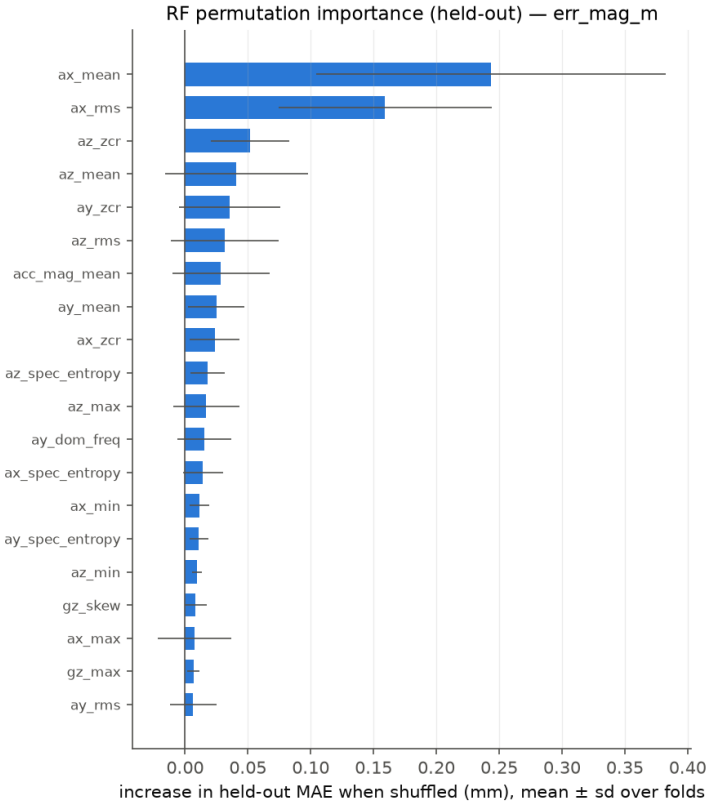


Figure 5: Random Forest permutation importance on held-out data: increase in MAE when one feature is shuffled, mean ± s.d. over the three folds.

remaining train/test gap is explained by the shift to MH_03, not by memorisation. More varied training flights (MH_04, MH_05) and models that see the raw 200×6 signal are the natural next steps for Phase 2.

References

[1] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[2] Michael Burri, Janosch Nikolic, Pascal Gohl, Thomas Schneider, Joern Rehder, Sammy Omari, Markus W. Achtelik, and Roland Siegwart. The EuRoC micro aerial vehicle datasets. *The International Journal of Robotics Research*, 35(10):1157–1163, 2016.